Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Web Service

Outline

- XML & JSON
- Asynchronous Process
- REST
- AJAX

XML & JSON

XML

- What is XML?
  - XML stands for eXtensible Markup Language
  - XML is a markup language much like HTML
  - XML was designed to store and transport data
  - XML was designed to be self-descriptive

```
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

JSON

- JSON: **J**ava**S**cript **O**bject **N**otation.
- JSON is a syntax for storing and exchanging data.
  - When exchanging data between a browser and a server, the data can only be text.
  - JSON is text, and we can convert any JavaScript object into JSON, and send JSON to the server.
  - We can also convert any JSON received from the server into JavaScript objects.
- Sending Data
  - ```
    var myObj = { "name":"John", "age":31, "city":"New York" };
    var myJSON = JSON.stringify(myObj);
    ```
- Receiving Data
  - ```
    var myJSON = '{ "name":"John", "age":31, "city":"New York" }';
    var myObj = JSON.parse(myJSON);
    ```

JSON Syntax

- JSON data is written as name/value pairs.
  - "name":"John"


```
{
    "message": "Berikut daftar kopi yang tersedia",
    "coffees": [
        {
            "id": 1,
            "name": "Kopi Tubruk",
            "price": 12000
        },
        {
            "id": 2,
            "name": "Kopi Tarik",
            "price": 15000
        },
        {
            "id": 3,
            "name": "Kopi Jawa",
            "price": 18000
        }
    ]
}
```


```
{"employees":[
    { "firstName":"John", "lastName":"Doe" },
    { "firstName":"Anna", "lastName":"Smith" },
    { "firstName":"Peter", "lastName":"Jones" }
]}
```

JSON vs XML

- JSON is Like XML Because
  - Both JSON and XML are "self describing" (human readable)
  - Both JSON and XML are hierarchical (values within values)
  - Both JSON and XML can be parsed and used by lots of programming languages
  - Both JSON and XML can be fetched with an XMLHttpRequest
- JSON is Unlike XML Because
  - JSON doesn't use end tag
  - JSON is shorter
  - JSON is quicker to read and write
  - JSON can use arrays
- The biggest difference is:
  - XML has to be parsed with an XML parser. JSON can be parsed by a standard JavaScript function.

Why JSON is Better Than XML

- XML is much more difficult to parse than JSON.
- JSON is parsed into a ready-to-use JavaScript object.
- For asynchronous applications, JSON is faster and easier than XML:

Using XML

- Fetch an XML document
- Use the XML DOM to loop through the document
- Extract values and store in variables

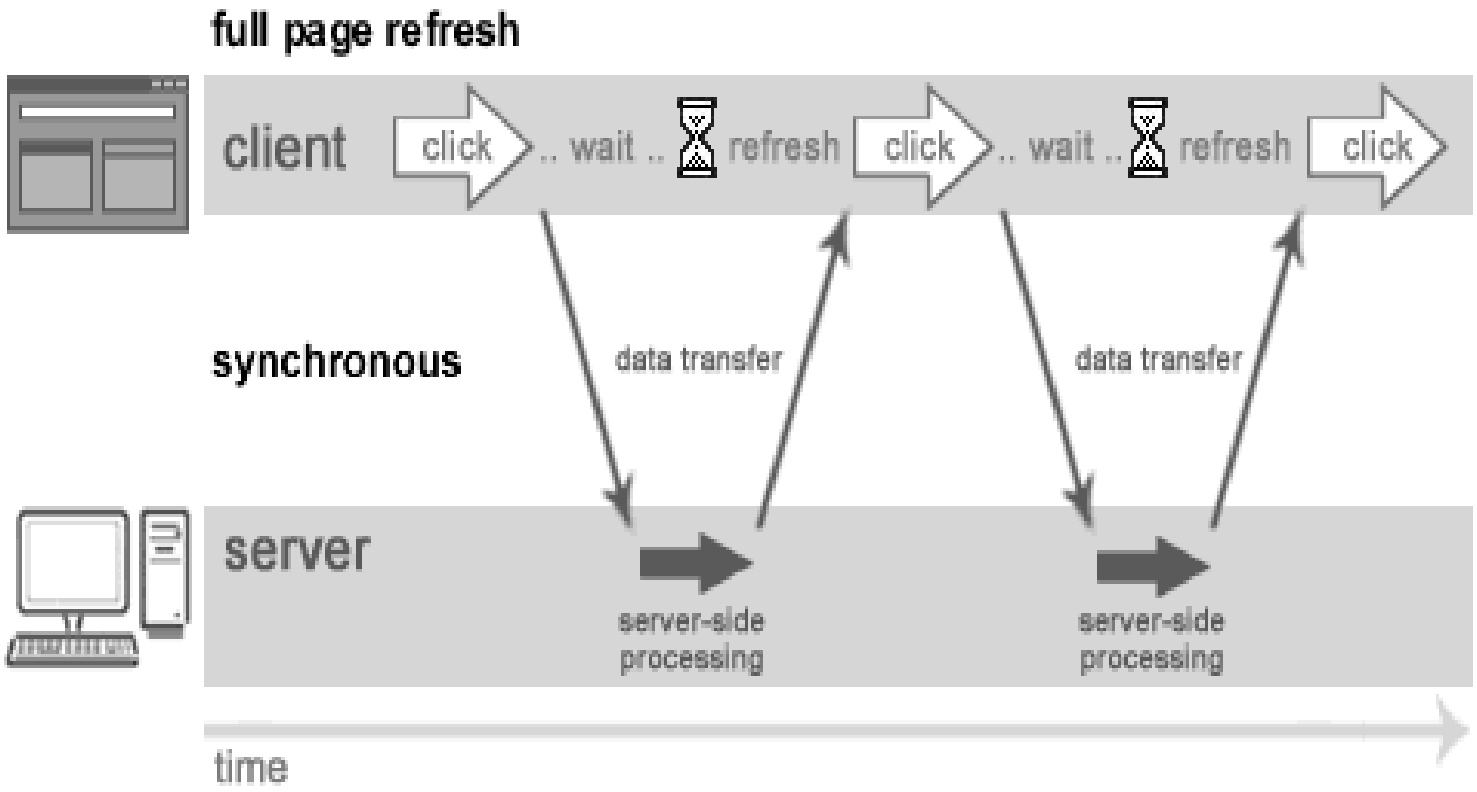
Using JSON

- Fetch a JSON string
- JSON.Parse the JSON string

Web Processing is Synchronous by default

- In computing, we say something is synchronous when it waits for an event to happen before continuing.
- A synchronous execution will block the web app until it finishes processing the current request.
- A disadvantage in this approach is that if a part of the code takes too long to execute, users have to wait and can’t continue because web app is blocked.
- Response will be relatively slower because page is loaded full again (like click F5 – refresh the browser)

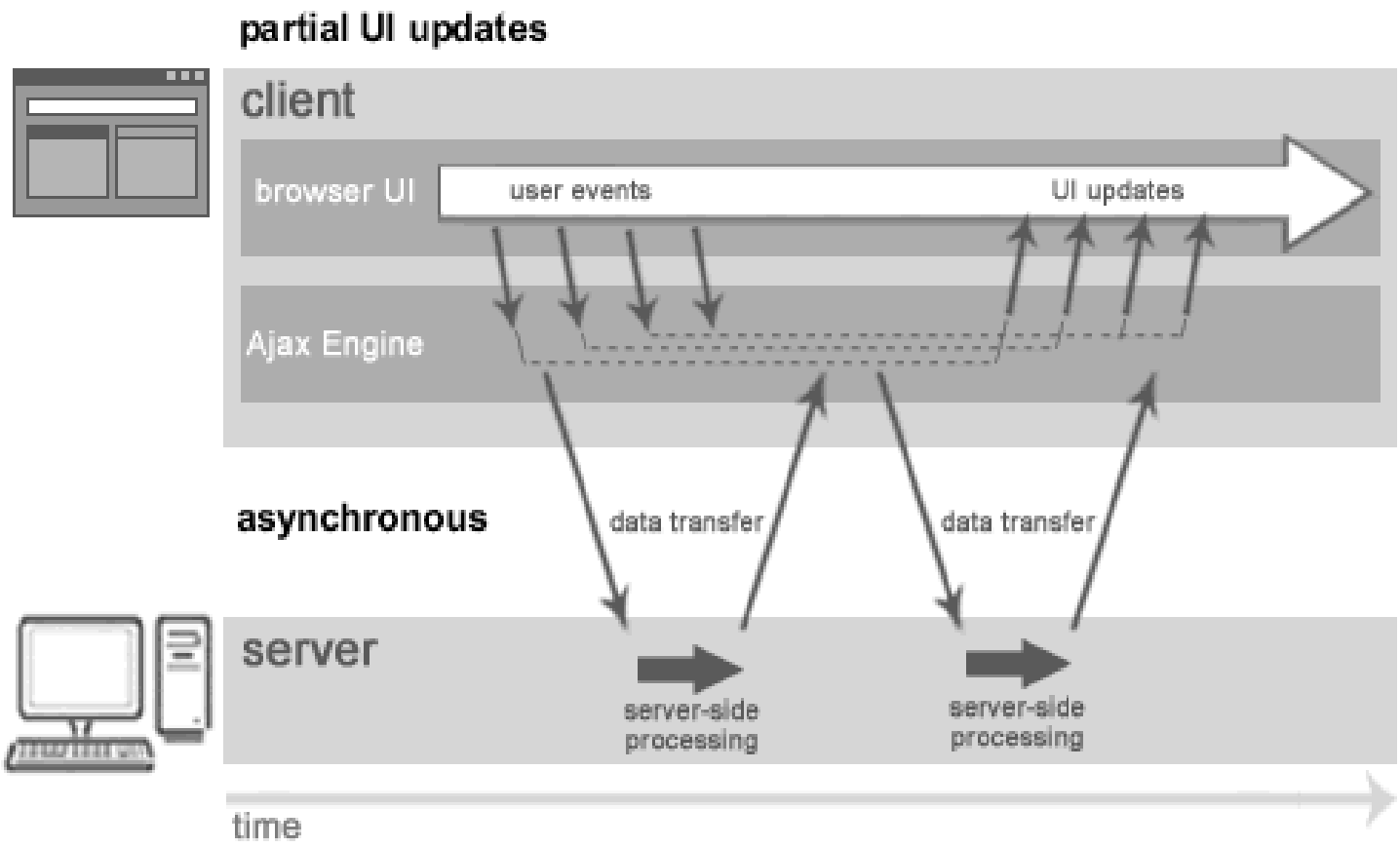
Synchronous Process



Asynchronous Process

- Asynchronous programming basically means no waiting or non-blocking programming model
- An asynchronous operation executes in a thread, separate from the main application thread
- When web app calls a method to perform an operation asynchronously, the web app can continue executing while the asynchronous method performs its task
- Response will be relatively faster because page is loaded partially (only the interface that changed)

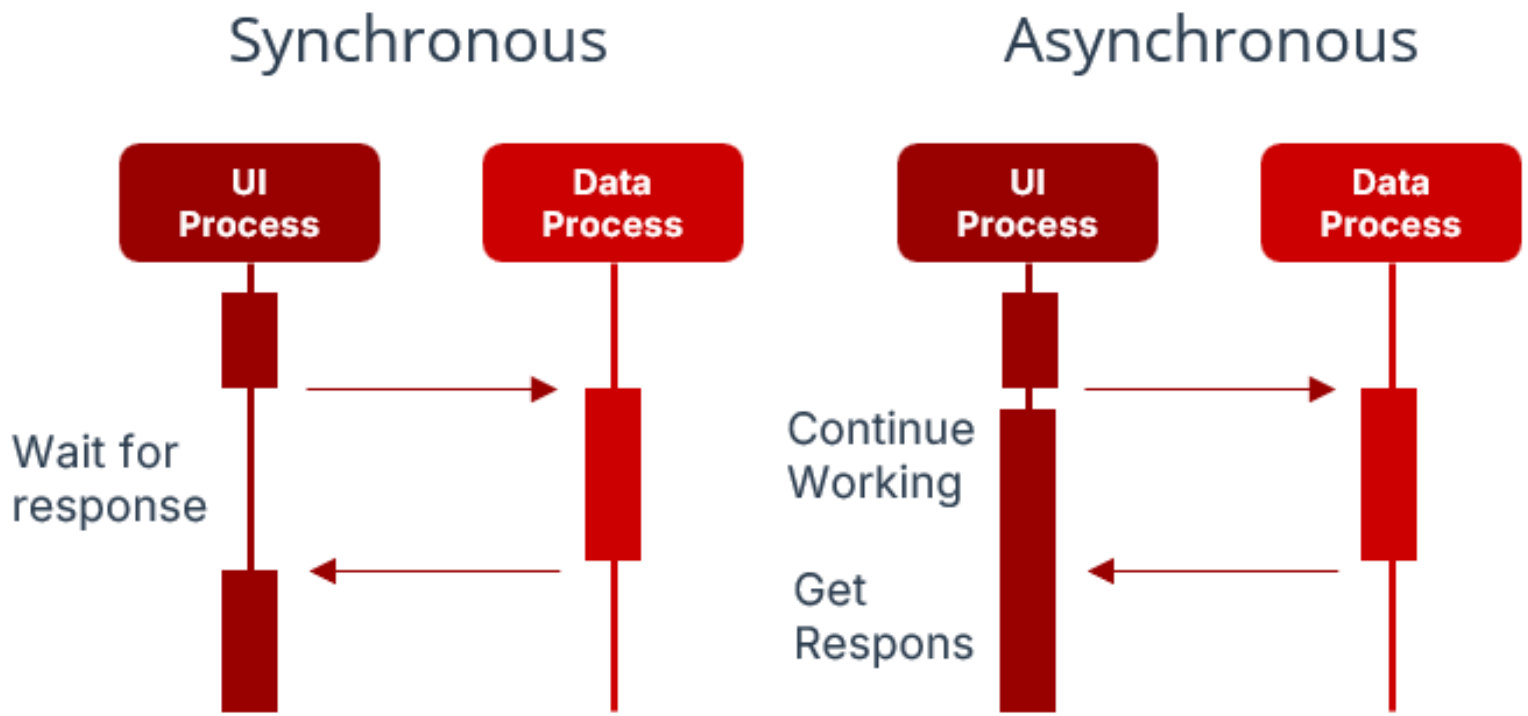
Asynchronous Process



Why asynchronous process?

- Network operations take time
- Avoiding blocking on the UI
- Efficient use of resources
- Ability to handle multiple requests simultaneously

Sync vs Async



Sync vs Async

Asynchronous Operation

- When doing an asynchronous operation, we can send many data formats:
  - Plain text, Binary, HTML, XML, JSON
- Asynchronous operation usually used HTTP API / Web Service
- There are many web service architectures
  - SOAP
  - RPC
  - REST
  - Web Socket
- Many web services today use REST

REST

- REST stands for REpresentational State Transfer, an architectural style for network communication between applications, which relies on a stateless protocol (usually HTTP) for interaction

- With the rise of mobile development and JavaScript frameworks, using a RESTful API is the best option to build a single interface between apps

REST

REST is commonly adopted for building web services because it has 4 good principles:

- Client-Server
- Stateless
  - Do not store client context between requests
- Cacheable
  - Can be used for improving performance
- Layered
  - The client does not need to be aware of the server's underlying architecture

RESTful API

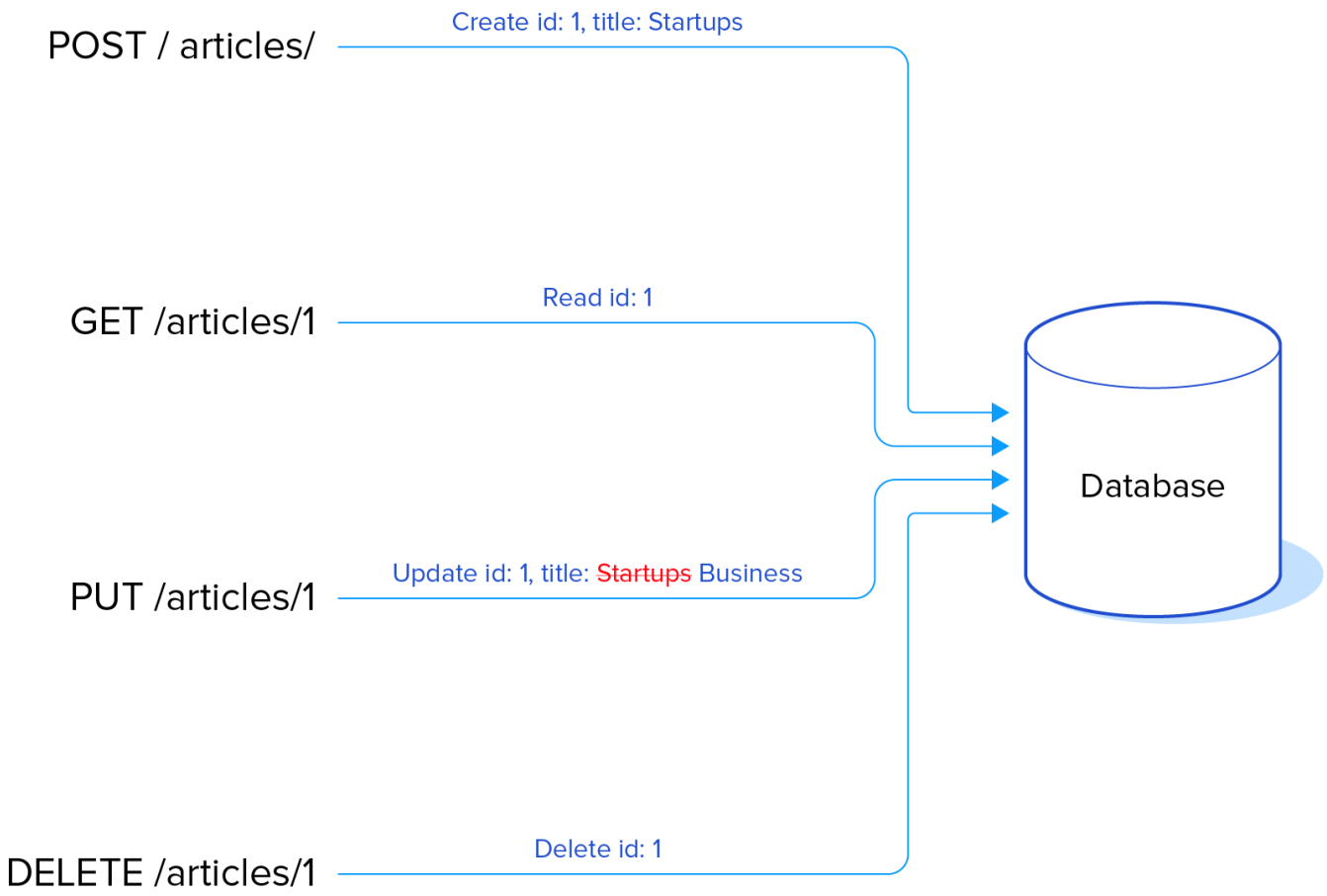
- An API that implements a REST architecture

RESTful API

- In RESTful APIs, we use the HTTP verbs as actions, and the endpoints are the resources acted upon. We'll be using the HTTP verbs for their semantic meaning:
  - GET: retrieve resources
  - POST: create resources
  - PUT: update resources
  - DELETE: delete resources

- Resources will be the targets of the actions and they have their own endpoints:
  - /articles
  - /users

REST


POST / articles/
Create id: 1, title: Startups
GET /articles/1
Read id: 1
PUT /articles/1
Update id: 1, title: Startups Business
DELETE /articles/1
Delete id: 1
Database

REST

- The greatest advantage of using a set of conventions such as REST is that your API will be much easier to consume and develop around

- Some endpoints are pretty straightforward and, as a result, your API will be much more easier to use and maintain, as opposed to having endpoints such as /get_article?id_article=12 or /delete_article?number=40

AJAX

AJAX

- Asynchronous Javascript & XML
- Enable client (browser) to communicate (do a request) with back-end server asynchronously
- Communication is done with a background process, and can return response as plain text, XML, JSON, or other formats
- AJAX is about loading data from server side in the background and display it on the webpage, without reloading the whole page

Two ways to perform AJAX

- XMLHttpRequest object
  - Traditional way, complex and callback-based syntax
- Fetch function
  - Modern way, easier to read and use

AJAX with jQuery

- https://api.jquery.com/jquery.ajax/

```
$.ajax({
    url:'url_data.json',
    global:false,
    async:false,
    dataType:'json',
    type:'post',
    data: ({ id : id }),
    success: function(e) {
        $.each(e, function(key, value) {
            $('#'+key).val(value);
        });
    },
    error : function() {
        alert('Error Guys..');
    },
    beforeSend : function() {
        $('#loading-img').show();
    },
    complete : function() {
        $('#loading-img').hide();
    }
});
```

AJAX with jQuery

OPTION	DESCRIPTION
url	The URL to make a request from
type	whether to use POST or GET
data	an object literal filled with query parameters and their values
dataType	The type of data you are expecting to recieve, one of: "text", "html", "json", "xml"
timeout	an amount of time in seconds to wait for the server before giving up
success	event: called when the request finishes successfully
error	event: called when the request fails
complete	event: called when the request finishes successfully or erroneously

AJAX with jQuery

- The data passed to your success handler will be in whatever format you specified in the dataType option
  - a dataType of text returns raw text no matter its apparent data type
  - a dataType of html returns raw html text
  - a dataType of xml returns an XML document object
  - a dataType of json returns a JSON object

Debugging AJAX code

Inspector | Console | Debugger | Style Editor | Performance | Memory | Network | Storage

All | HTML | CSS | JS | XHR | Fonts | Images | Media | WS | Other | Persist Logs | Disable cache | Filter URLs

St...	M...	File	Domain	Ca...	T...	Tra...	Si...	0 ms / 10.24 s / 20.48 s / 30.7
304	GET	pace.min.js	localhost	script	js	cached	12.21 KB	→ 35 ms
304	GET	jquery.min.js	localhost	script	js	cached	82.37 KB	→ 0 ms
304	GET	bootstrap.min.js	localhost	script	js	cached	36.19 KB	→ 0 ms
304	GET	blockui.min.js	localhost	script	js	cached	8.95 KB	→ 0 ms
304	GET	datatables.min...	localhost	script	js	cached	81.27 KB	→ 10 ms
304	GET	select2.min.js	localhost	script	js	cached	73.25 KB	→ 13 ms
304	GET	validate.min.js	localhost	script	js	cached	22.18 KB	→ 6 ms
304	GET	app.js	localhost	script	js	cached	18.60 KB	→ 6 ms
304	GET	setting.js	localhost	script	js	cached	7.47 KB	→ 1 ms
304	GET	ripple.min.js	localhost	script	js	cached	5.05 KB	→ 1 ms
304	GET	logo_light.png	localhost	img	png	cached	7 KB	→ 1 ms
304	GET	video1.png	localhost	img	png	cached	205.73 KB	→ 2 ms
304	GET	user_bg4.jpg	localhost	img	jpeg	cached	172.66 KB	→ 2 ms
200	POST	json	localhost	xhr	html	14.86 KB	14.33 KB	→ 57 ms
200	GET	components.css	localhost	stylesh...	css	cached	351.31 KB	
200	GET	bootstrap.css	localhost	stylesh...	css	cached	112.99 KB	

26 requests | 2.04 MB / 1.43 MB transferred | Finish: 27.87 s | DOMContentLoaded: 607 ms | load: 705 ms

Headers | Cookies | Params | Response | Timings | Stack Trace

Filter properties

JSON

```
draw: 1
recordsTotal: 1733
recordsFiltered: 1733
data: [...]
  0: [...]
    0: 2017/05/27 10:53<br><span class="text-grey-300"><em><i class="fa fa-edit"></i>
       2017/07/06 23:01</em></span>
    1: Agro Wisata Desa Dangiang
    2: destinasi-wisata
    3: Publish
    4: <a href="pwadmin/posts/form/1" title="Edit Data"><span class="label label-
       primary label-table"><i class="fa fa-edit"></i> Edit</span></a> <a
       href="javascript:publish(1,0)" title="Unpublish"><span class="label label-danger
       label-table"><i class="fa fa-eye-slash"></i> Unpublish</span></a>
  1: [...]
    0: 2017/05/27 10:53<br><span class="text-grey-300"><em><i class="fa fa-edit"></i>
       2017/07/04 15:24</em></span>
    1: Agro Wisata Desa Pasirwangi
```

Example (client)

```
...
<form id="form1">
  <input type="text" name="msg" id="msg" />
  <input type="button" value="Shout!" id="shoutbutton" />
</form>
<div id="data"></div>
<script type="text/javascript" src="jquery.js"></script>
<script type="text/javascript" src="ajax.js"></script>
...
```

Example (ajax.js)

```
$(document).ready(function(){
  $("#shoutbutton").click(function() {
    $.ajax({
      type: "GET", //change to "POST" for POST request
      url: "reply.php",
      data: $("#form1").serialize(),
      success: function(rsp) {
        $("#data").append("<div>"+rsp+"</div>");
      },
      error: function(rsp) {
        alert(rsp);
      }
    });
  });
});
```

Example (server) - reply.php

```
<?php
  if (isset($_GET["pesan"])) {
     echo "reply: ".$_GET["pesan"];
  }
?>
```

Any question?

References

- https://www.w3schools.com/js/js_json_intro.asp
- https://www.w3schools.com/js/js_ajax_intro.asp

Fakultas Informatika
School of Computing
Telkom University

THANK YOU